Récupérer des données avec des API depuis Python

Lino Galiana
Version du 16 octobre 2026

Les **API** (*Application Programming Interface*) sont un mode d'accès aux données en expansion. Grâce aux API, l'automatisation de scripts est facilitée puisqu'il n'est plus nécessaire de stocker un fichier, et gérer ses différentes versions, mais uniquement de requêter une base et laisser au producteur de données le soin de gérer les mises à jour de la base.

⚠️ **PDF généré automatiquement.** Ce document est produit à partir du site du cours et peut présenter des problèmes de mise en forme. Pour une version à jour et adaptée à la lecture, consultez la page du cours : pythonds.linogaliana.fr.

Table des matières

1 Introduction : Qu'est-ce qu'une API ? 1
Approche pédagogique 3
2 Première utilisation d'API 3
2.1 Comprendre le principe avec un exemple interactif 3
2.2 Comment faire avec `Python` ? 3
2.3 Comment connaître les *inputs* et *outputs* des API ? 5
2.4 Application 5
3 Plus d'exemples de requêtes `GET` 6
3.1 Source principale 6
3.2 Récupérer des données à façon grâce aux API 7
4 Découverte des requêtes `POST` 9
4.1 Logique 9
4.2 Principe 9
4.3 Application avec `Python` 9
5 Gestion des secrets et des exceptions 10
5.1 Utiliser un token dans un code sans le révéler 11
5.2 Comprendre le principe avec une documentation interactive 12
5.3 Récupération des données via `Python` 12
6 Ouverture aux API de modèles 13
7 Exercices supplémentaires 14
Informations additionnelles 16

1 Introduction : Qu'est-ce qu'une API ?

Nous avons vu dans les chapitres précédents comment consommer des données depuis un fichier (le mode d'accès le plus simple) ou comment récupérer des données par le biais du *webscraping*, une méthode qui permet à `Python` de singer le comportement d'un navigateur web et de récupérer de l'information en moissonnant le HTML auquel accède un site web.

Le webscraping est un pis-aller pour accéder à de la donnée. Heureusement, il existe d'autres manières d'accéder à des données : les API de données. En informatique, une API est un ensemble de protocoles permettant à deux logiciels de communiquer entre eux. Par exemple, on parle parfois

d'API Pandas ce qui désigne le fait que `Pandas` est une interface entre votre code `Python` et un langage compilé plus efficace (`C`) qui fait les calculs que vous demandez au niveau de `Python`. L'objectif d'une API est ainsi de fournir un point d'accès à une fonctionnalité qui soit facile à utiliser et qui masque les détails de la mise en oeuvre.

Dans ce chapitre, nous nous intéressons principalement aux API de données. Ces dernières sont simplement une façon de mettre à disposition des données : plutôt que de laisser l'utilisateur consulter directement des bases de données (souvent volumineuses et complexes), l'API lui propose de formuler une requête qui est traitée par le serveur hébergeant la base de données, puis de recevoir des données en réponse à sa requête.

L'utilisation accrue d'API dans le cadre de stratégies open-data est l'un des piliers des 15 feuilles de route ministérielles françaises en matière d'ouverture, de circulation et de valorisation des données publiques.

> **Note**
>
> Depuis quelques années, un service officiel de géocodage a été mis en place pour le territoire français. Celui-ci est gratuit et permet de manière efficace de coder des adresses à partir d'une API. Cette API, connue sous le nom de la **Base d'Adresses Nationale (BAN)** a bénéficié de la mise en commun de données de plusieurs acteurs (collectivités locales, Poste, IGN) et de compétences d'acteurs comme Etalab. La documentation de celle-ci est disponible à l'adresse https://www.data.gouv.fr/dataservices/api-adresse-base-adresse-nationale-ban/.

On prend souvent comme exemple pour illustrer les API l'exemple du restaurant. La documentation est votre menu : elle liste les plats (les bases de données) que vous pouvez commander et les éventuels ingrédients de celle-ci que vous pouvez choisir (les paramètres de votre requête) : poulet, boeuf ou option végé ? Lorsque vous faites ceci, vous ne connaissez pas la recette utilisée en arrière cuisine pour faire votre plat: vous recevez seulement celui-ci. De manière logique, plus le plat que vous avez demandé est raffiné (calculs complexes côté serveur), plus votre plat mettra du temps à vous arriver.

> **Illustration avec l'API BAN**
>
> Pour illustrer ceci, imaginons ce qu'il se passe lorsque, dans la suite du chapitre, nous ferons des requêtes à l'API BAN.
>
> Via `Python`, on envoie notre commande à celle-ci: des adresses plus ou moins complètes avec des instructions annexes comme le code commune. Ces instructions annexes peuvent s'apparenter à des informations fournies au serveur du restaurant comme des interdits alimentaires qui vont personnaliser la recette.
>
> A partir de ces instructions, le plat est lancé. En l'occurrence, il s'agit de faire tourner sur les serveurs d'Etalab une routine qui va chercher dans un référentiel d'adresse celle qui est la plus similaire à celle qu'on a demandé en adaptant éventuellement en fonction des instructions annexes qu'on a fourni. Une fois que côté cuisine on a fini cette préparation, on renvoie le plat au client. En l'occurrence, le plat sera des coordonnées géographiques qui correspondent à l'adresse la plus similaire.
>
> Le client n'a donc qu'à se préoccuper de faire une bonne requête et d'apprécier le plat qui lui est fourni. L'intelligence dans la mise en oeuvre est laissée aux spécialistes qui ont conçu l'API. Peut-être que d'autres spécialistes, par exemple Google Maps, mettent en oeuvre une recette différente pour ce même plat (des coordonnées géographiques) mais ils vous proposeront probablement un menu très similaire. Ceci vous simplifie beaucoup la vie: il vous suffit de changer quelques lignes de code d'appel à une API plutôt que de modifier un ensemble long et complexe de méthodes d'identification d'adresses.

Approche pédagogique

Après une première présentation du principe général des API, ce chapitre illustre l'usage de celles-ci via `Python` par le biais d'un *use case* assez standard: on dispose d'un jeu de données qu'on désire d'abord géolocaliser. Pour cela, on va demander à une API de nous renvoyer des coordonnées géographiques à partir d'adresses. Ensuite on ira chercher des informations un peu plus complexes par le biais d'autres API.

2 Première utilisation d'API

Une API a donc vocation à servir d'intermédiaire entre un client et un serveur. Ce client peut être de deux types: une interface web ou un logiciel de programmation. L'API ne fait pas d'*a priori* sur l'outil qui sert lui passe une commande, elle lui demande seulement de respecter un standard (en général une requête http), une structure de requête (les arguments) et d'attendre le résultat.

2.1 Comprendre le principe avec un exemple interactif

Le premier mode (accès par un navigateur) est principalement utilisé lorsqu'une interface web permet à un utilisateur de faire des choix afin de lui renvoyer des résultats correspondant à ceux-ci. Prenons à nouveau l'exemple de l'API de géolocalisation que nous utiliserons dans ce chapitre. Imaginons une interface web permettant à l'utilisateur deux choix: un code postal et une adresse. Cela sera injecté dans la requête et le serveur répondra avec la géolocalisation adaptée.

2.2 Comment faire avec `Python` ?

Le principe est le même sauf que nous perdons l'aspect interactif. Il s'agira donc, avec `Python`, de construire l'URL voulue et d'aller chercher via une requête HTTP le résultat.

Nous avons déjà vu dans le chapitre de webscraping la manière dont `Python` communique avec internet: via le *package* `requests`. Ce *package* suit le protocole HTTP où on retrouve principalement deux types de requêtes: `GET` et `POST`:

- La requête `GET` est utilisée pour récupérer des données depuis un serveur web. C'est la méthode la plus simple et courante pour accéder aux ressources d'une page web. Nous allons commencer par décrire celle-ci.
- La requête `POST` est utilisée pour envoyer des données au serveur, souvent dans le but de créer ou de mettre à jour une ressource. Sur les pages web, elle sert souvent à la soumission de formulaires qui nécessitent de mettre à jour des informations sur une base (mot de passe, informations clients, etc.). Nous verrons son utilité plus tard, lorsque nous commencerons à rentrer dans les requêtes authentifiées où il faudra soumettre des informations supplémentaires à notre requête.

Faisons un premier test avec `Python` en faisant comme si nous connaissions bien cette API.

```
import requests
adresse = "88 avenue verdier"
url_ban_example  =  f"https://data.geopf.fr/geocodage/search/?q={adresse.replace("  ",  "+")}
&postcode=92120"
requests.get(url_ban_example)
```

```
<Response [200]>
```

Qu'est-ce qu'on obtient ? Un code HTTP. Le code 200 correspond aux requêtes réussies, c'est-à-dire pour lesquelles le serveur est en mesure de répondre. Si ce n'est pas le cas, pour une raison x ou y, vous aurez un code différent.

> Les codes HTTP
>
> Les codes de statut HTTP sont des réponses standard envoyées par les serveurs web pour indiquer le résultat d'une requête effectuée par un client (comme un navigateur ou un script Python). Ils sont classés en différentes catégories selon le premier chiffre du code :
>
> - 1xx : Informations
> - 2xx : Succès
> - 3xx : Redirections
> - 4xx : Erreurs côté client
> - 5xx : Erreurs côté serveur
>
> Ceux à retenir sont : 200 (succès), 400 (requête mal structurée), 401 (authentification non réussie), 403 (accès interdit), 404 (ressource demandée n'existe pas), 503 (le serveur n'est pas en capacité de répondre)

Pour récupérer le contenu renvoyé par `requests`, il existe plusieurs méthodes. Quand on un JSON bien formatté, le plus simple est d'utiliser la méthode `json` qui transforme cela en dictionnaire :

```
req = requests.get(url_ban_example)
localisation_insee = req.json()
localisation_insee
```


```
{'type': 'FeatureCollection',
 'features': [{'type': 'Feature',
   'geometry': {'type': 'Point', 'coordinates': [2.309144, 48.81622]},
   'properties': {'label': '88 Avenue Verdier 92120 Montrouge',
    'score': 0.9734354545454544,
    'housenumber': '88',
    'id': '92049_9625_00088',
    'banId': '92dd3c4a-6703-423d-bf09-fc0412fb4f89',
    'name': '88 Avenue Verdier',
    'postcode': '92120',
    'citycode': '92049',
    'x': 649270.67,
    'y': 6857572.24,
    'city': 'Montrouge',
    'context': '92, Hauts-de-Seine, Île-de-France',
    'type': 'housenumber',
    'importance': 0.70779,
    'depcode': '92',
    'street': 'Avenue Verdier',
    '_type': 'address'}}],
 'query': '88 avenue verdier'}
```


En l'occurrence, on voit que les données sont dans un JSON imbriqué. Il faut donc développer un peu de code pour récupérer les informations voulues dans celui-ci:

```
localisation_insee.get('features')[0].get('properties')
```

```
{'label': '88 Avenue Verdier 92120 Montrouge',
 'score': 0.9734354545454544,
 'housenumber': '88',
 'id': '92049_9625_00088',
 'banId': '92dd3c4a-6703-423d-bf09-fc0412fb4f89',
 'name': '88 Avenue Verdier',
 'postcode': '92120',
 'citycode': '92049',
 'x': 649270.67,
 'y': 6857572.24,
 'city': 'Montrouge',
 'context': '92, Hauts-de-Seine, Île-de-France',
 'type': 'housenumber',
 'importance': 0.70779,
 'depcode': '92',
 'street': 'Avenue Verdier',
 '_type': 'address'}
```

C'est là l'inconvénient principal de l'usage des API: le travail *ex post* sur les données renvoyées. Le code nécessaire est propre à chaque API puisque l'architecture du JSON dépend de chaque API.

2.3 Comment connaître les *inputs* et *outputs* des API ?

Ici on a pris l'API BAN comme un outil magique dont on connaissait les principaux *inputs* (l'*endpoint*, les paramètres et leur formattage...). Mais comment faire, en pratique, pour en arriver là ? Tout simplement en lisant la documentation lorsqu'elle existe et en testant celle-ci via des exemples.

Les bonnes API proposent un outil interactif qui s'appelle le `swagger`. C'est un site web interactif où sont décrites les principales fonctionnalités de l'API et où l'utilisateur peut tester des exemples interactivement. Ces documentations sont souvent créées automatiquement lors de la construction d'une API et mises à disposition par le biais d'un point d'entrée `/docs`. Elles permettent souvent d'éditer certains paramètres dans le navigateur, voir le JSON obtenu (ou l'erreur générée) et récupérer la requête formattée qui permet d'obtenir celui-ci. Ces consoles interactives dans le navigateur permettent de répliquer le tâtonnement qu'on peut faire par ailleurs dans des outils spécialisés comme `postman`.

Concernant l'API BAN, la documentation se trouve sur https://www.data.gouv.fr/dataservices/api-adresse-base-adresse-nationale-ban/. Si vous cliquez sur « search » puis « Try it out! », vous aurez accès à la version interactive. La documentation présente de nombreux exemples qui peuvent être testés directement depuis le navigateur, génère l'URL de la requête, vous donne le code réponse et la réponse. Vous pouvez aussi utiliser les URL proposées comme exemple en utilisant `curl` (un équivalent de `requests` en ligne de commande Linux):

```
curl "https://data.geopf.fr/geocodage/search/?q=8+bd+du+port&limit=15"
```

Il suffit de copier l'URL en question (`https://data.geopf.fr/geocodage/search/?q=8+bd+du+port&limit=15`), d'ouvrir un nouvel onglet et vérifier que cela produit bien un résultat. Puis de changer un paramètre et vérifier à nouveau, jusqu'à trouver la structure qui convient. Et après, on peut passer à `Python` comme le propose l'exercice suivant.

2.4 Application

Pour cet exercice, vous aurez besoin de la variable suivante:

```
adresse = "88 Avenue Verdier"
```

> Exercice 1: Structurer un appel à une API depuis Python
>
> 1. Tester sans aucun autre paramètre, le retour de notre API. Transformer en `DataFrame` le résultat.
> 2. Se restreindre à Montrouge avec le paramètre *ad hoc* et la recherche du code insee ou code postal adéquat sur google.
> 3. (Optionnel): Représenter l'adresse trouvée sur une carte

Les deux premières lignes du *dataframe* obtenu à la question 1 devraient être

	label	score	housenumber	id	banId	name	postcode	citycode	x	y	city	context	type	importance	depcode	street	_type	locality
0	88 Avenue Verdier 92120 Montrouge	0.973435	88	92049_9625_00088	92dd3c4a-6703-423d-bf09-fc0412fb4f89	88 Avenue Verdier	92120	92049	649270.67	6857572.24	Montrouge	92, Hauts-de-Seine, Île-de-France	housenumber	0.70779	92	Avenue Verdier	address	NaN
1	Avenue Verdier 44500 La Baule-Escoublac	0.719494	NaN	44055_3690	e6e5569-c6ba-49fc-bbf8-863301a8b2df	Avenue Verdier	44500	44055	291895.65	6701236.88	La Baule-Escoublac	44, Loire-Atlantique, Pays de la Loire	street	0.60193	44	Avenue Verdier	address	NaN

A la question 2, on ressort cette fois qu'une seule observation, qu'on pourrait retravailler avec `GeoPandas` pour vérifier qu'on a bien placé ce point sur une carte

	label	score	housenumber	id	banId	name	postcode	citycode	x	y	city	context	type	importance	depcode	street	_type	geometry
0	88 Avenue Verdier 92120 Montrouge	0.973435	88	92049_9625_00088	92dd3c4a-6703-423d-bf09-fc0412fb4f89	88 Avenue Verdier	92120	92049	649270.67	6857572.24	Montrouge	92, Hauts-de-Seine, Île-de-France	housenumber	0.70779	92	Avenue Verdier	address	POINT (2.30914 48.81622)

Enfin, à la question 3, on obtient cette carte (plus ou moins la même que précédemment):

3 Plus d'exemples de requêtes `GET`

3.1 Source principale

Nous allons utiliser comme base principale pour ce tutoriel la base permanente des équipements, un répertoire d'équipements publics accueillant du public.

On va commencer par récupérer les données qui nous intéressent. On ne récupère pas toutes les variables du fichier mais seulement celles qu'ils nous intéressent: quelques variables sur l'équipement, son adresse et sa commune d'appartement.

Nous allons nous restreindre aux établissements d'enseignement primaire, secondaire et supérieur du département de la Haute-Garonne (le département 31). Ces établissements sont identifiés par un code particulier, entre `C1` et `C5`.

```
import duckdb

query = """
FROM read_parquet('https://minio.lab.sspcloud.fr/lgaliana/diffusion/BPE23.parquet')
SELECT NOMRS, NUMVOIE, INDREP, TYPVOIE, LIBVOIE,
       CADR, CODPOS, DEPCOM, DEP, TYPEQU,
       concat_ws(' ', NUMVOIE, INDREP, TYPVOIE, LIBVOIE) AS adresse, SIRET
WHERE DEP = '31'
      AND NOT (starts_with(TYPEQU, 'C6') OR starts_with(TYPEQU, 'C7'))
"""

bpe = duckdb.sql(query)
bpe = bpe.to_df()
bpe = bpe[bpe['TYPEQU'].str.startswith('C')]
```

```
FloatProgress(value=0.0, layout=Layout(width='auto'), style=ProgressStyle(bar_color='black'))
```

3.2 Récupérer des données à façon grâce aux API

Nous avons vu précédemment le principe général d'une requête d'API. Pour illustrer, de manière plus massive, la récupération de données par le biais d'une API, essayons de récupérer des données complémentaires à notre source principale. Nous allons utiliser l'annuaire de l'éducation qui fournit de nombreuses informations sur les établissements scolaires. Nous utiliserons le SIRET pour croiser les deux sources de données.

L'exercice suivant viendra illustrer l'intérêt d'utiliser une API pour avoir des données à façon et la simplicité à récupérer celles-ci via `Python`. Néanmoins, cet exercice illustrera également une des limites de certaines API, à savoir la volumétrie des données à récupérer.

> Exercice 2
>
> 1. Visiter le *swagger* de l'API de l'Annuaire de l'Education nationale sur data.gouv.fr et tester une première récupération de données en utilisant le *endpoint* `records` sans aucun paramètre.
> 2. Puisqu'on n'a conservé que les données de la Haute Garonne dans notre base principale, on désire ne récupérer que les établissements de ce département par le biais de notre API. Faire une requête avec le paramètre *ad hoc*, sans en ajouter d'autres.
> 3. Augmenter la limite du nombre de paramètres, voyez-vous le problème ?
> 4. Les métadonnées de la réponse permettent de savoir le total de réponse. En analysant finement la documentation de l'API, voyez-vous comment nous pourrions récupérer toutes les données automatiquement ?

La première question nous permet de récupérer un premier jeu de données

	identifiant_de_l_etablissement	nom_etablissement	type_etablissement	statut_public_prive	adresse_1	adresse_2	adresse_3	code_postal	code_commune	nom_commune	...	libelle_nature	code_type_contrat_prive	pial	etablissement_mere	type_rattachement_etablissement_mere	code_circonscription	code_zone_animation_pedagogique	libelle_zone_animation_pedagogique	code_bassin_formation	libelle_bassin_formation
0	0030754F	Ecole élémentaire Ampère	Ecole	Public	25 rue Ampère	NaN	01460 YZEURE	01460	03321	Yzeure	...	ECOLE DE NIVEAU ELEMENTAIRE	99	0030604L	None	None	0030604D	NaN	NaN	0601I	MOULINS
1	0030765N	Ecole maternelle Emile Zola	Ecole	Public	16 rue Managran	NaN	03100 MONTLUCON	03100	03185	Montluçon	...	ECOLE MATERNELLE	99	0030030S	None	None	0030066F	NaN	NaN	0603I	MONTLUCON

Néanmoins, on a deux problèmes : il n'y a que 10 observations et elles couvrent toute la France et pas juste notre département d'intérêt. Essayons déjà avec la question 2 de changer ce dernier.

	identifiant_de_l_etablissement	nom_etablissement	type_etablissement	statut_public_prive	adresse_1	adresse_2	adresse_3	code_postal	code_commune	nom_commune	...	libelle_nature	code_type_contrat_prive	pial	etablissement_mere	type_rattachement_etablissement_mere	code_circonscription	code_zone_animation_pedagogique	libelle_zone_animation_pedagogique	code_bassin_formation	libelle_bassin_formation
0	0310163H	Ecole primaire d'application Bénéfet	Ecole	Public	37 rue Bénéfet	None	31089 TOULOUSE	31389	31555	Toulouse	...	ECOLE ELEMENTAIRE D APPLICATION	99	0310053X	None	None	0312042D	None	None	1612G	TOULOUSE OUEST
1	0310171T	Ecole maternelle publique Jean Macé	Ecole	Public	6 square François Lahille	None	31770 COLOMIERS	31770	31169	Colomiers	...	ECOLE MATERNELLE	99	0311152X	None	None	0312191J	None	None	1612T	TOULOUSE NORD-OUEST
2	0310174T	Ecole maternelle publique Le Courrasus	Ecole	Public	Rue du Courrasus	None	31210 MONTREJEAU	31210	31390	Montréjeau	...	ECOLE MATERNELLE	99	0310095M	None	None	0312066X	None	None	1610G	COMMINGES
3	0310192E	Ecole primaire publique La résidence	Ecole	Public	Rue de la Résidence	None	31088 ST GAUDENS	31088	31483	Saint-Gaudens	...	ECOLE DE NIVEAU ELEMENTAIRE	99	0310085X	None	None	0311105L	None	None	1610G	COMMINGES
4	0310199K	Ecole maternelle publique Henri Guillaumet	Ecole	Public	14 chemin de Bibet	None	31400 TOULOUSE	31400	31555	Toulouse	...	ECOLE MATERNELLE	99	0310085Z	None	None	0312023C	None	None	1619I	TOULOUSE-CENTRE

C'est mieux, mais nous avons toujours seulement 10 observations. Si on essaie d'ajuster le nombre de lignes (question 3), on obtient le retour suivant de l'API :

```
b'{\n  "error_code": "InvalidRESTParameterError",\n  "message": "Invalid value for limit API
parameter: 200 was found but -1 ⩽ limit ⩽ 100 is expected."\n}'
```

En regardant les métadonnées de la réponse envoyée par l'API, on voit qu'il existe une variable `total_count` qui permet de donner la taille totale du dataset (1269 lignes en l'occurence). Pour extraire toutes les données, on va utiliser la variable `offset`, qui permet de décaler la réponse envoyée par l'API et de récupérer à partir de la n^{ième} ligne. Comme le code d'automatisation est assez fastidieux à écrire, le voici :

```
import requests
import pandas as pd

# Initialize the initial API URL
dep = '031'
offset = 0
limit = 100
url_api_datagouv = f'https://data.education.gouv.fr/api/v2/catalog/datasets/fr-en-annuaire-
education/records?where=code_departement=\'{dep}\'&limit={limit}&offset={offset}'
```

```
# First request to get total obs and first df
response = requests.get(url_api_datagouv)
nb_obs = response.json()['total_count']
schools_dep31 = pd.json_normalize(response.json()['records'])

# Loop on the nb of obs
while nb_obs > len(schools_dep31):
    try:
        # Increase offset for first reply sent and update API url
        offset += limit
            url_api_datagouv  =  f'https://data.education.gouv.fr/api/v2/catalog/datasets/fr-en-
annuaire-education/records?where=code_departement=\'{dep}\'&limit={limit}&offset={offset}'

        # Call API
        print(f'fetching from {offset}th row')
        response = requests.get(url_api_datagouv)

        # Concatenate the data from this call to previous data
        page_data = pd.json_normalize(response.json()['records'])
        schools_dep31 = pd.concat([schools_dep31, page_data], ignore_index=True)
        print(f'length of data is now {len(schools_dep31)}')

    except requests.exceptions.RequestException as e:
        print(f"An error occurred: {e}")
        break

# Filter columns to keep only those starting with 'record.fields'
schools_dep31 = schools_dep31.filter(regex='^record\\.fields\\..*')

# Rename columns by removing 'record.fields.' at the beginning
schools_dep31 = schools_dep31.rename(columns=lambda x: x.replace('record.fields.', ''))
```

Le *DataFrame* obtenu est le suivant:

```
schools_dep31.head()
```

	identifiant_de_l_etablissement	nom_etablissement	type_etablissement	statut_public_prive	adresse_1	adresse_2	adresse_3	code_postal	code_commune	nom_commune	...	code_type_contrat_prive	pial	etablissement_mere	type_rattachement_etablissement_mere	code_circonscription	code_zone_animation_pedagogique	libelle_zone_animation_pedagogique	code_bassin_formation	libelle_bassin_formation	position
0	0311616H	Ecole primaire d'application Bénazet	Ecole	Public	37 rue Bénazet	NaN	31300 TOULOUSE	31300	31555	Toulouse	...	99	0310037X	None	None	0312626D	None	None	16126	TOULOUSE OUEST	NaN
1	0311617T	Ecole maternelle publique Jean Macé	Ecole	Public	6 square François Labille	NaN	31770 COLOMIERS	31770	31149	Colomiers	...	99	0311535X	None	None	0312141J	None	None	16127	TOULOUSE NORD-OUEST	NaN
2	0311676V	Ecole maternelle publique Le Courrasse	Ecole	Public	Rue du Courrasse	NaN	31210 MONTREJEAU	31210	31390	Montréjeau	...	99	0310005M	None	None	0312866X	None	None	16106	COMMINGES	NaN
3	0311918E	Ecole primaire publique La résidence	Ecole	Public	Rue de la Résidence	NaN	31400 ST GAUDENS	31400	31483	Saint-Gaudens	...	99	0310003X	None	None	0311198L	None	None	16106	COMMINGES	NaN
4	0311619X	Ecole maternelle publique Henri Guillaumet	Ecole	Public	14 chemin de Rémi	NaN	31400 TOULOUSE	31400	31555	Toulouse	...	99	0310005Z	None	None	0312825C	None	None	16109	TOULOUSE CENTRE	NaN

On peut fusionner ces nouvelles données avec nos données précédentes pour enrichir celles-ci. Pour faire une production fiable, il faudrait faire attention aux écoles qui ne s'apparient pas, mais ce n'est pas grave pour cette série d'exercices.

```
bpe_enriched = bpe.merge(
  schools_dep31,
  left_on = "SIRET",
  right_on = "siren_siret"
)
bpe_enriched.head(2)
```

	NOMRS	NUMVOIE	INDREP	TYPVOIE	LIBVOIE	CADR	CODPOS	DEPCOM	DEP	TYPEQU	...	code_type_contrat_prive	pial	etablissement_mere	type_rattachement_etablissement_mere	code_circonscription	code_zone_animation_pedagogique	libelle_zone_animation_pedagogique	code_bassin_formation	libelle_bassin_formation	position
0	ECOLE PRIMAIRE PUBLIQUE DENIS LATAPIE			LD	LA BOURDETTE		31230	31001	31	C108	...	99	0310005K	None	None	0311104L	None	None	16106	COMMINGES	NaN
1	ECOLE MATERNELLE PUBLIQUE	21		CHE	DE L AUTAN		31280	31003	31	C107	...	99	0311535H	None	None	0311102E	None	None	16128	TOULOUSE EST	NaN

Cela nous donne des données enrichies de nouvelles caractéristiques sur les établissements. Il y a des coordonnées géographiques dans celles-ci, mais nous allons faire comme s'il n'y en avait pas pour réutiliser notre API de géolocalisation.

4 Découverte des requêtes `POST`

4.1 Logique

Nous avons jusqu'à présent évoqué les requêtes `GET`. Nous allons maintenant présenter les requêtes `POST` qui permettent d'interagir de manière plus complexe avec des serveurs de l'API.

Pour découvrir celles-ci, nous allons reprendre l'API de géolocalisation précédente mais utiliser un autre point d'entrée qui nécessite une requête `POST`.

Ces dernières sont généralement utilisées quand il est nécessaire d'envoyer des données particulières pour déclencher une action. Par exemple, dans le monde du web, si vous avez une authentification à mettre en oeuvre, une requête `POST` permettra d'envoyer un *token* au serveur qui répondra en acceptant votre authentification.

Dans notre cas, nous allons envoyer des données au serveur, ce dernier va les recevoir, les utiliser pour la géolocalisation puis nous envoyer une réponse. Pour continuer sur la métaphore culinaire, c'est comme si vous donniez vous-mêmes à la cuisine un *tupperware* pour récupérer votre plat à emporter.

4.2 Principe

Prenons cette requête proposée sur le site de documentation de l'API de géolocalisation:

```
curl -X POST -F data=@path/to/file.csv -F columns=voie -F columns=ville -F
citycode=ma_colonne_code_insee https://data.geopf.fr/geocodage/search/csv
```

Comme nous avons pu l'évoquer précédemment, `curl` est un outil en ligne de commande qui permet de faire des requêtes API. L'option `-X POST` indique, de manière assez transparente, qu'on désire faire une requête `POST`.

Les autres arguments sont passés par le biais des options `-F`. En l'occurrence, on envoie un fichier et on ajoute des paramètres pour aider le serveur à aller chercher la donnée dedans. L'`@` indique que `file.csv` doit être lu sur le disque et envoyé dans le corps de la requête comme une donnée de formulaire.

4.3 Application avec `Python`

Nous avions `requests.get`, il est donc logique que nous ayons `requests.post`. Cette fois, il faudra passer des paramètres à notre requête sous la forme d'un dictionnaire dont les clés sont le nom de l'argument et les valeurs sont des objets `Python`.

Le principal défi, illustré dans le prochain exercice, est le passage de l'argument `data`: il faudra renvoyer le fichier comme un objet `Python` par le biais de la fonction `open`.

Exercice 3: une requête POST pour géolocaliser en masse nos données

1. Enregistrer au format CSV les colonnes `adresse`, `DEPCOM` et `nom_commune` de la base d'équipements fusionnée avec notre répertoire précédent (objet `bpe_enriched`). Il peut être utile, avant l'écriture au format CSV, de remplacer les virgules dans la colonne `adresse` par des espaces.
2. Créer l'objet `response` avec `requests.post` et les bons arguments pour géocoder votre CSV.
3. Transformer votre output en objet `geopandas` avec la commande suivante:

```
bpe_loc = pd.read_csv(io.StringIO(response.text))
```

Les géolocalisations obtenues prennent cette forme

	index	adresse	DEPCOM	nom_commune	result_score	latitude	longitude
0	0	LD LA BOURDETTE	31001	Agassac	0.404457	43.374288	0.880679
1	1	21 CHE DE L AUTAN	31003	Aigrefeuille	0.730598	43.567370	1.585932

En enrichissant les données précédentes, cela donne:

	NOMRS	NUMVOIE	INDREP	TYPVOIE	LIBVOIE	CADR	CODPOS	DEPCOM	DEP	TYPEQU	...	type_rattachement_etablissement_mere	code_circonscription	code_zone_animation_pedagogique	libelle_zone_animation_pedagogique	code_bassin_formation	libelle_bassin_formation	position	result_score	latitude_ban	longitude_ban
0	ECOLE PRIMAIRE PUBLIQUE DENIS LATAPIE			LD	LA BOURDETTE		31250	31001	31	C108	...	None	0311108L	None	None	16106	COMMINGES	NaN	0.404457	43.374288	0.880679
1	ECOLE MATERNELLE PUBLIQUE	21		CHE	DE L AUTAN		31260	31003	31	C107	...	None	0311102E	None	None	16128	TOULOUSE EST	NaN	0.730598	43.567370	1.585932

On peut vérifier que la géolocalisation ne soit pas trop délirante en comparant avec les longitudes et latitudes de l'annuaire de l'éducation ajouté précédemment:

	NOMRS	nom_commune	longitude_annuaire	longitude_ban	latitude_annuaire	latitude_ban
307	SECTION ENSEIGNT PROFESSIONNEL LPO SIMONE DE B...	Gragnague	1.578128	1.580267	43.692174	43.690664
870	ECOLE ELEMENTAIRE PUBLIQUE BONNEFOY	Toulouse	1.454510	1.454298	43.616635	43.616935
59	ECOLE MATERNELLE PUBLIQUE	Beaupuy	1.553909	1.554426	43.646302	43.646480
517	ECOLE ELEMENTAIRE PUBLIQUE DU RUISSEAU	Pinsaguel	1.389980	1.390127	43.505863	43.506730
19	ECOLE MATERNELLE PUBLIQUE JOSEPHINE BAKER	Aurignac	0.881090	0.880754	43.215974	43.216656

Sans rentrer dans le détail, les positions semblent très similaires à quelques imprécisions près.

Pour profiter de nos données enrichies, on peut faire une carte. Pour ajouter un peu de contexte à celle-ci, on peut mettre un fond de carte des communes en arrière plan. Celui-ci peut être récupéré avec `cartiflette`:

```
from cartiflette import carti_download
shp_communes = carti_download(
  crs = 4326,
  values = ["31"],
  borders="COMMUNE",
  filter_by="DEPARTEMENT",
  source="EXPRESS-COG-CARTO-TERRITOIRE",
  year=2022
)
shp_communes.crs = 4326
```

```
/__w/python-datascientist/python-datascientist/.venv/lib/python3.13/site-packages/pandas/core/
generic.py:6208: FutureWarning: Overriding the CRS of a GeoDataFrame that already has CRS. This
unsafe behavior will be deprecated in future versions. Use GeoDataFrame.set_crs method instead
  return object.__setattr__(self, name, value)
```

Représentées sur une carte, cela donne la carte suivante:

5 Gestion des secrets et des exceptions

Nous avons déjà utilisé plusieurs API. Néanmoins ces dernières étaient toutes sans authentification et présentent peu de restrictions, hormis le nombre d'échos. Ce n'est pas le cas de toutes les API. Il est fréquent que les API qui permettent d'aspirer plus de données ou d'accéder à des données confidentielles nécessitent une authentification pour tracer les utilisateurs de données.

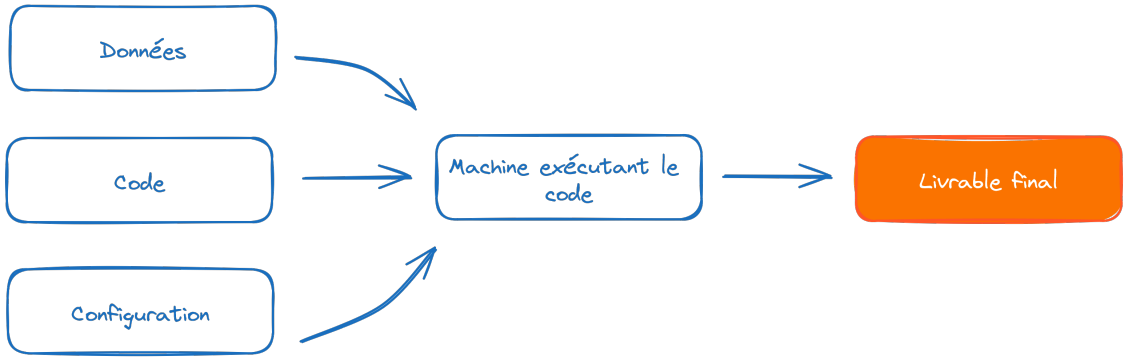
Cela se fait généralement par le biais d'un *token*. Ce dernier est une sorte de mot de passe, souvent utilisé dans les systèmes modernes d'authentification pour certifier de l'identité d'un utilisateur.trice (cf. chapitre `Git`).

Pour illustrer l'usage des *tokens*, nous allons utiliser une API de l'Insee. Nous allons utiliser cette API pour récupérer des informations supplémentaires sur nos établissements à partir de leur code SIREN.

Avant d'en arriver là, nous allons faire un aparté sur la confidentialité des tokens et la manière d'éviter de révéler ceux-ci dans votre code.

5.1 Utiliser un token dans un code sans le révéler

Les tokens sont des informations personnelles qui ne doivent pas être partagées. Ils n'ont pas vocation à être présent dans le code. Comme ceci est évoqué à plusieurs reprises dans le cours de mise en production que Romain Avouac et moi donnons en 3e année, il est important de séparer le code des éléments de configuration



L'idée est de trouver une recette pour apporter les éléments de configuration avec le code mais sans mettre ceux-ci en clair dans le code. L'idée générale sera de stocker la valeur du *token* dans une variable mais ne jamais révéler celle-ci dans le code. Comment faire dès lors pour déclarer la valeur du jeton sans que celui-ci soit apparent dans le code ?

- Pour un code amené à fonctionner de manière interactive (par exemple par le biais d'un *notebook*), il est possible de créer une boite de dialogue qui injectera la valeur renseignée dans une variable. Cela se fait par le biais du package `getpass`.
- Pour le code qui tourne en non interactif, par exemple par le biais de la ligne de commande, l'approche par variable d'environnement est la plus fiable, à condition de faire attention à ne pas mettre le fichier de mot de passe dans `Git` . Pour cela, le plus simple est d'utiliser `dotenv` si vous faites tourner votre code ou des secrets si votre code tourne de manière automatisée sur des serveurs *cloud*.

Il ne faut jamais mettre de *token* dans `Git`. Sinon, vous courrez le risque d'avoir votre identité usurpée : des robots scannent en continu `Github` à la recherche de jetons pour ensuite lancer des dénis de service en se faisant passer pour vous.

Si vous avez partagé par erreur un jeton : pas de panique, cela peut arriver ! L'avantage des jetons est qu'ils sont révocables : vous pouvez l'invalider et en créer un nouveau pour continuer à utiliser le service désiré. La bonne réaction consiste à révoquer le jeton le plus vite possible, une fois la fuite constatée. La meilleure parade pour éviter ce type de fuite est d'ajouter tout de suite le `.env` au `.gitignore`.

L'exercice suivant permettra de découvrir comment ajouter de manière confidentielle un *payload* à des requêtes d'authentification, c'est-à-dire des informations confidentielles identifiantes en complément d'une requête.

5.2 Comprendre le principe avec une documentation interactive

Pour illustrer l'utilisation des API authentifiées, nous proposons d'explorer le portail des API de l'Insee.

Nous allons nous concentrer sur l'API Sirene mais, sur ce portail, il en existe d'autres, notamment l'API Melodi consacrée à la récupération d'un certain nombre de sources *open data* de l'Insee. L'API Sirene est une version interrogeable des données Sirene *open data*.

Avant d'aller dans `Python`, utilisons notre navigateur pour comprendre le principe de l'authentification.

Exercice 4 : API authentifiée par le biais du navigateur

1. Se créer un compte sur le portail des API
2. Aller voir l'espace `Mes applications` et en créer une nouvelle. Par simplicité, vous pouvez la nommer *« Python data science »*. Au cours de la création de cette application, souscrire à l'API Sirene.
3. Cliquer sur l'onglet *Souscriptions*. Choisir l'API Sirene qui devrait normalement être disponible dans le bas de la page.
4. A droite, vous devriez maintenant voir s'afficher un jeton. Gardez cette page ouverte et ouvrez dans un nouvel onglet portail-api.insee.fr/catalog/all.

Testons maintenant l'API Sirene grâce au *swagger* en partant de portail-api.insee.fr/catalog/all

1. Cliquer sur l'onglet `Documentation`. La documentation interactive (le *swagger*) doit maintenant s'afficher.
2. Plus bas, dans la documentation interactive se rendre au point d'entrée `/siren/{siren}` (méthode `GET`).
3. Cliquer sur le bouton `Try it out` pour essayer l'exemple de manière interactive.
4. Remplir le champ `siren` avec le SIREN `500569405` qui est l'identifiant de Décathlon 😉

Si vous avez l'erreur ci-dessous, comprenez-vous pourquoi ?

```
//| echo: false
error_request_no_token
```

Il va falloir être authentifié.

1. Pour cela, recharger cette documentation interactive et cliquer sur le bouton `Authorize`. Dans le *popup* qui s'affiche, renseigner le jeton issu de l'autre fenêtre.
2. Après avoir validé ce jeton, retentez la procédure précédente de récupération des informations de Décathlon.

Vous devriez maintenant avoir cette sortie :

```
//| echo: false
sortie_sirene_decathlon
```

5.3 Récupération des données via `Python`

Pour cette application, à partir de la question 4, nous allons avoir besoin de créer une classe spéciale permettant à `requests` de surcharger notre requête d'un jeton d'authentification. Comme elle n'est pas triviale à créer sans connaissance préalable, la voici:

```
class BearerAuth(requests.auth.AuthBase):
    def __init__(self, token):
        self.token = token
    def __call__(self, r):
        r.headers["X-INSEE-Api-Key-Integration"] = self.token
        return r
```

```
siren = "500569405"
```

Exercice 5 : les tokens avec Python

1. Créer une variable `token` par le biais de `getpass`.
2. Utiliser cette structure de code pour récupérer la donnée voulue

```
requests.get(
    url,
    auth=BearerAuth(token)
)
```

3. Remplacer l'utilisation de `getpass` par l'approche variable d'environnement grâce à `dotenv`.

! Important

L'approche par variable d'environnement est la plus générale et malléable. Il faut néanmoins bien faire attention à ne pas oublier d'ajouter le `.env`, qui stocke des identifiants, dans le fichier `.gitignore`. Autrement, vous risquez de révéler des informations identifiantes ce qui annule tout effet positif des bonnes pratiques mises en oeuvre avec `dotenv`.

Pour cela, la solution est simple : ajouter la ligne `.env` au `.gitignore` et par sécurité `*.env` au cas où le fichier ne soit pas à la racine du dépôt. Pour en savoir plus sur ce fichier `.gitignore`, se rendre sur les chapitres `Git`.

6 Ouverture aux API de modèles

Nous avons vu jusqu'à présent des API de données. Celles-ci permettent de récupérer du code. Ce n'est néanmoins pas la seule utilisation des API intéressantes pour les utilisateurs de `Python`.

Il existe de nombreux autres types d'API. Parmi celles-ci, les API de modèles sont intéressantes. Elles permettent de récupérer des modèles pré-entraînés voire effectuer une phase d'inférence sur des serveurs spécialisés ayant plus de ressources que son ordinateur local (plus d'éléments dans les parties *machine learning* et NLP). La librairie la plus connue dans ce domaine est la librairie `transformers` développée par `HuggingFace`.

L'un des objectifs du cours de 3A de mise en production est de montrer comment ce type d'architecture logicielle fonctionne et comment celle-ci peut être créée sur des modèles que vous auriez vous-mêmes créés.

7 Exercices supplémentaires

Exercice bonus 1: et si on ajoutait des informations sur la valeur ajoutée des lycées ?

Dans notre exemple sur les écoles, se restreindre aux lycées et ajouter les informations sur la valeur ajoutée des lycées disponibles ici.

Exercice bonus 2: on sort où ce soir ?

Trouver un lieu commun où se retrouver entre amis est toujours l'objet d'âpres négociations: et si on laissait guider par la géographie ?

1. Créer un `DataFrame` enregistrant une série d'adresses et de codes postaux comme l'exemple ci-dessous
2. Adapter le code de l'exercice sur l'API BAN, avec l'appui de la documentation, pour géolocaliser ces adresses
3. En supposant que vos données géolocalisées se nomment `adresses_geocoded`, utiliser le code proposé pour transformer celles-ci en polygone
4. Calculer le centroid et représenter sur une carte interactive `Folium` comme précédemment

Vous aviez oublié qu'il y avait un couple dans le groupe... Tenir compte de la variable `poids` pour calculer le barycentre et trouver où vous retrouver ce soir.

Créer le polygone à partir des géolocalisations

```
from shapely.geometry import Polygon
coordinates = list(zip(adresses_geocoded["longitude"], adresses_geocoded["latitude"]))
polygon = Polygon(coordinates)

polygon = gpd.GeoDataFrame(index=[0], crs="epsg:4326", geometry=[polygon])
polygon
```

Le DataFrame d'exemple:

```
adresses_text = pd.DataFrame(
  {
    "adresse": [
        "10 Rue de Rivoli",
        "15 Boulevard Saint-Michel",
        "8 Rue Saint-Honoré",
        "20 Avenue des Champs-Élysées",
        "Place de la Bastille",
    ],
    "cp": ["75004", "75005", "75001", "75008", "75011"],
    "poids": [2, 1, 1, 1, 1]
})
adresses_text
```

	adresse	cp	poids
0	10 Rue de Rivoli	75004	2
1	15 Boulevard Saint-Michel	75005	1
2	8 Rue Saint-Honoré	75001	1
3	20 Avenue des Champs-Élysées	75008	1

	adresse	cp	poids
4	Place de la Bastille	75011	1

	adresse	poids	cp	result_score	latitude	longitude
0	10 Rue de Rivoli	2	75004	0.970158	48.855500	2.360410
1	15 Boulevard Saint-Michel	1	75005	0.973296	48.851852	2.343614
2	8 Rue Saint-Honoré	1	75001	0.866200	48.863801	2.335725
3	20 Avenue des Champs-Élysées	1	75008	0.872216	48.871049	2.303730
4	Place de la Bastille	1	75011	0.965214	48.853711	2.370213

La géolocalisation obtenue pour cet exemple

Voici la carte obtenue sur le jeu d'exemple. On sera peut-être plus au centre avec le barycentre qu'avec le centroid.

```
apiroot = "https://data.geopf.fr/geocodage"
param1 = {
  const AdresseFormat = adresse.toLowerCase().replaceAll(" ", "+")
  const url = `q=${AdresseFormat}`
  return url
}
param2 = `postcode=${codePostal}`
```

```
import {mj} from "@danielefadda/mathjax"
```

```
url = {
  const AdresseFormat = adresse.toLowerCase().replaceAll(" ", "+")
 const url = `https://data.geopf.fr/geocodage/search/?q=${AdresseFormat}&postcode=${codePostal}
`
  return url
}
```

```
localisation = d3.json(url)
```

```
defaultAdresse = "88 Avenue Verdier"
longitude = localisation.features[0].geometry.coordinates[0]
latitude = localisation.features[0].geometry.coordinates[1]
```

```
map = {
  const container = html`<div style="height:300px;">`;
  yield container;
  const map = L.map(container).setView([latitude, longitude], 13);
  L.tileLayer("https://{s}.tile.openstreetmap.org/{z}/{x}/{y}.png", {
      attribution: "&copy; <a href=https://www.openstreetmap.org/copyright>OpenStreetMap</a>
contributors"
  }).addTo(map);
  var marker = L.marker([latitude, longitude]).addTo(map);
  marker.bindPopup("<b>Trouvé !</b>").openPopup();
  return map
}
```

```
import {L} from "@observablehq/hello-leaflet"
```

```
<IPython.core.display.HTML object>
```

Informations additionnelles

i Environnement Python

Ce site a été construit automatiquement par le biais d'une action `Github` utilisant le logiciel de publication reproductible `Quarto` (version 1.10.18).

L'environnement utilisé pour obtenir les résultats est reproductible par le biais d'uv. Le fichier `pyproject.toml` utilisé pour construire cet environnement est disponible sur le dépôt `linogaliana/python-datascientist`

```
[project]
name = "python-datascientist"
version = "0.1.0"
description = "Source code for Lino Galiana's Python for data science course"
readme = "README.md"
requires-python = "≥3.13,<3.14"
dependencies = [
    "altair≥6.0.0",
    "cartiflette≥0.2",
    "contextily==1.6.2",
    "duckdb≥0.10.1",
    "folium≥0.19.6",
    "gdal==3.11.4",
    "graphviz==0.20.3",
    "great-tables≥0.12.0",
    "gt-extras≥0.0.8",
    "ipykernel≥6.29.5",
    "jupyter≥1.1.1",
    "jupyter-cache≥1.0.0",
    "kaleido≥0.2.1",
    "langchain-community≥0.3.27",
    "loguru==0.7.3",
    "markdown≥3.8",
    "nbclient≥0.10.0",
    "nbformat≥5.10.4",
    "nltk≥3.9.1",
    "openpyxl≥3.1.5",
    "pandas≥3.0",
    "pip≥25.1.1",
    "plotly≥6.1.2",
    "plotnine≥0.15",
    "polars≥1.8.2",
    "py7zr≥1.1.3",
    "pyarrow≥17.0.0",
    "pynsee≥0.1.8",
    "python-dotenv≥1.0.1",
    "python-frontmatter≥1.1.0",
    "pywaffle≥1.1.1",
    "requests≥2.32.3",
    "scikit-image≥0.24.0",
    "scikit-learn≥1.8.0",
    "scipy≥1.13.0",
```

```
    "seaborn>=0.13.2",
    "selenium<4.39.0",
    "spacy>=3.8.4",
    "webdriver-manager>=4.0.2",
    "wordcloud==1.9.3",
]

[tool.uv.sources]
gdal = [
  { index = "gdal-wheels", marker = "sys_platform == 'linux'" },
  { index = "geospatial_wheels", marker = "sys_platform == 'win32'" },
]

[[tool.uv.index]]
name = "geospatial_wheels"
url = "https://nathanjmcdougall.github.io/geospatial-wheels-index/"
explicit = true

[[tool.uv.index]]
name = "gdal-wheels"
url = "https://gitlab.com/api/v4/projects/61637378/packages/pypi/simple"
explicit = true

[dependency-groups]
dev = [
    "nb-clean>=4.0.1",
]
```

Pour utiliser exactement le même environnement (version de `Python` et *packages*), se reporter à la documentation d'`uv`.